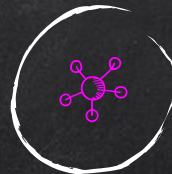


# APPLICATION PROGRAMMING INTERFACE

- Les API en généraux
- API REST :
  - Définition d'un API REST
  - Le protocole HTTP
  - Les méthodes et les codes de retour d'une API REST
  - Concevoir une API REST sur .NET
    - Création d'un projet API sur .NET
    - Structure du projet
    - Les contrôleurs
    - Les décorateurs (Controller, méthodes et arguments)
    - Les méthodes GET, POST, PUT, DELETE
    - Notions du pattern DTO
    - Gestion des versions d'une API REST
    - Documentation
      - Pourquoi documenter une API REST
      - Documentation automatique avec swagger
    - Sécurité :
      - Sécurité des API REST
      - Implémentation de JWT



# API

## APPLICATION PROGRAMMING INTERFACE

*Ces couches permettent de faire une abstraction du code côté serveur, et de fournir aux clients un choix exhaustifs de fonctionnalités.*

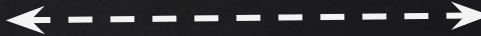
Il existe plusieurs types d'API :

- Les API RPC (REMOTE CALL PROCEDURE) : GRPC
- Les API WEB : SOAP, REST, WEBSockets, SIGNAL-R
- ...



API

PROTOCOLE (HTTP, TCP/IP, ...)



Sérialisation et désérialisation  
des objets



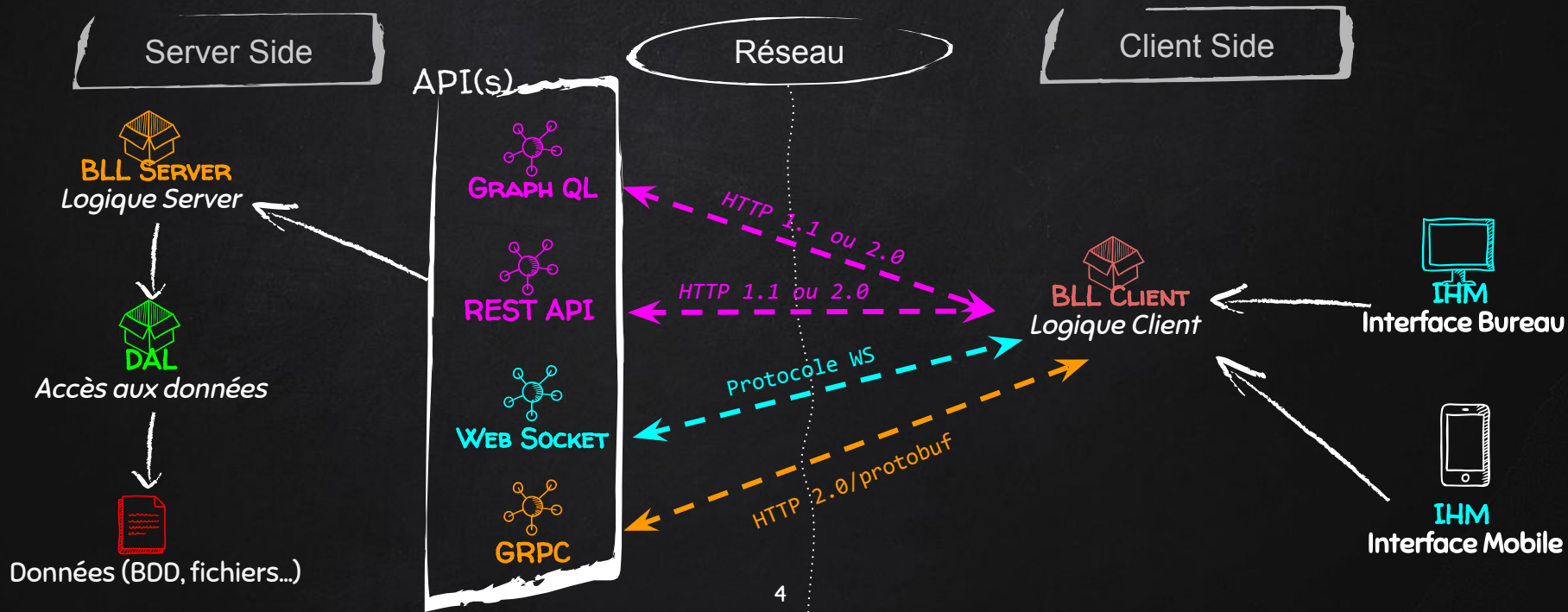
BLL CLIENT  
Logique Client

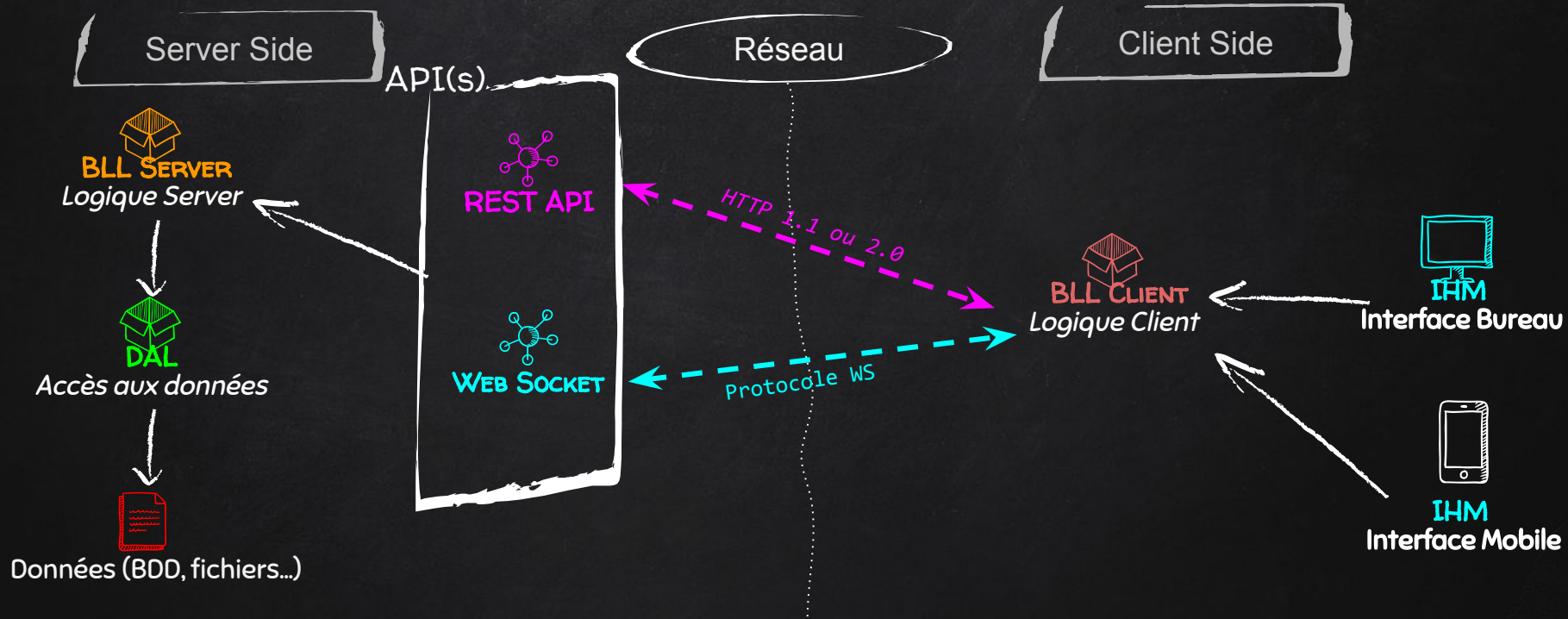


Une API est un **contrat**  
entre les clients et le  
serveur !

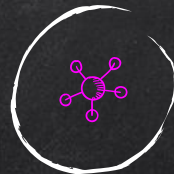


# ARCHITECTURE AVEC API CLIENT / SERVER N TIERS





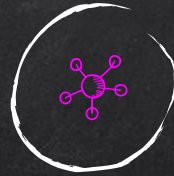




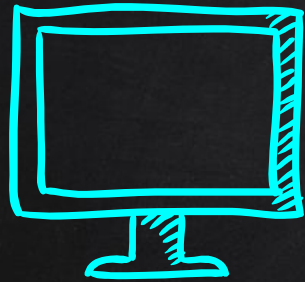
# API REST

## REPRESENTATIONAL STATE TRANSFER

- Une API REST est **sans état (stateless)** c'est à dire que la communication entre le client et le serveur ne doit pas dépendre d'un quelconque contexte provenant du serveur.
- La requête doit **contenir l'ensemble des informations nécessaires à son traitement.**
- Elle est **composée d' EndPoints** dont certains sont des **URI (Uniform Resource Identifier)**
- **Chaque URI est unique**



# PROTOCOLE HTTP



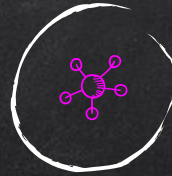
CLIENT

HTTP Request

HTTP Response

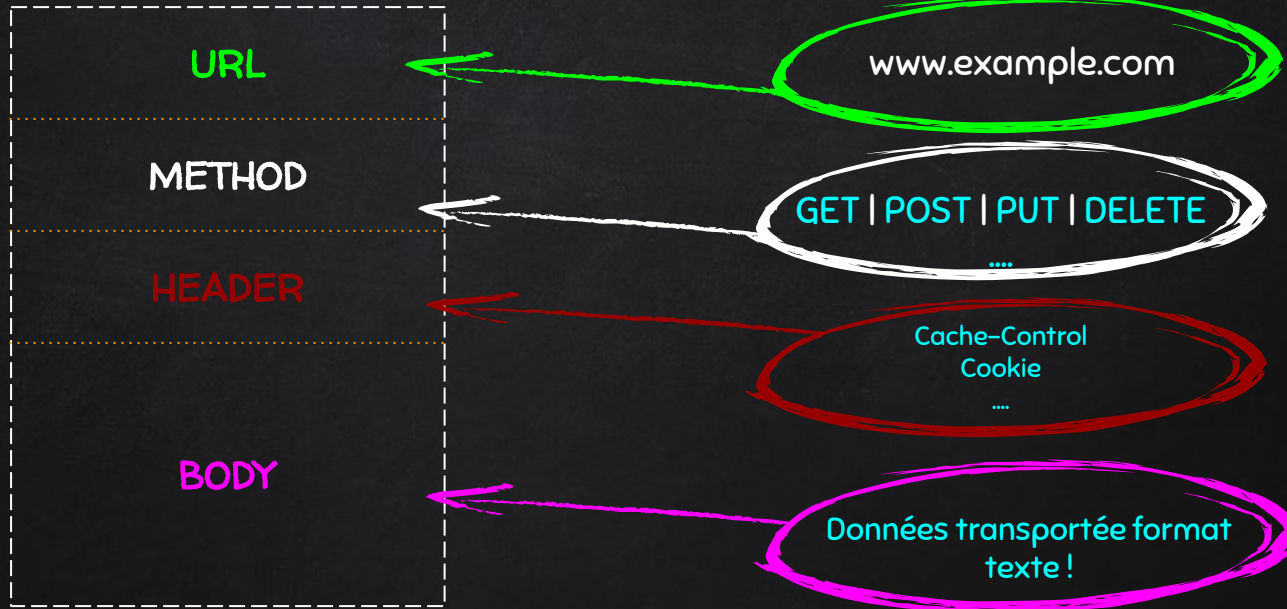
SERVER



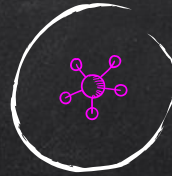


# PROTOCOLE HTTP

## Les requêtes

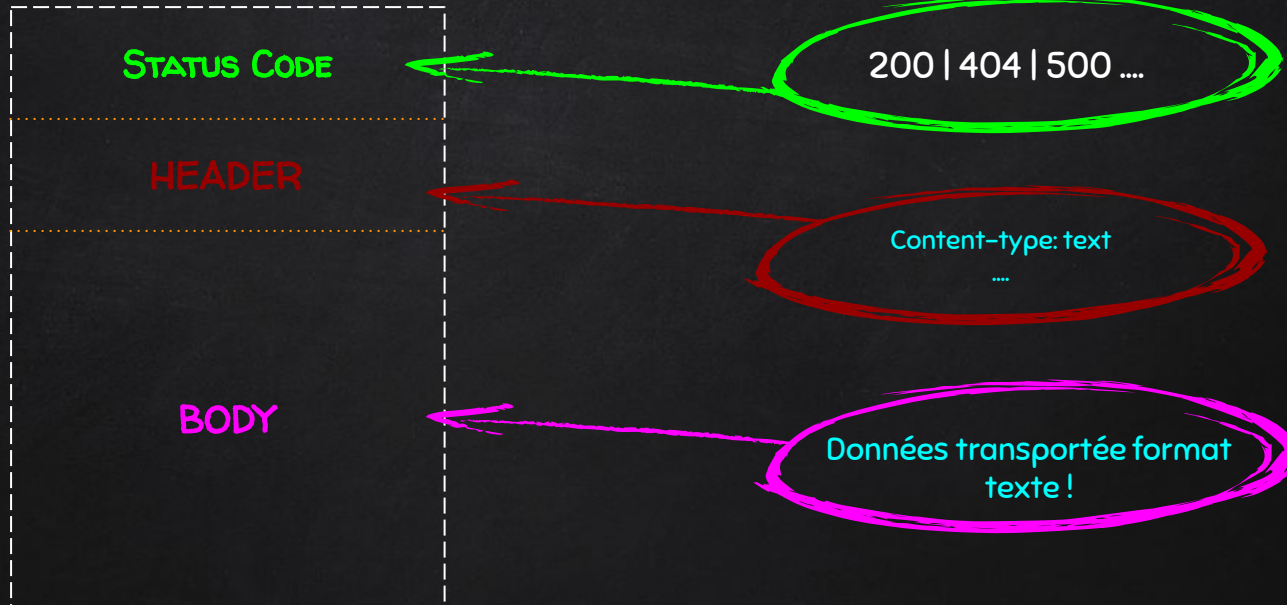


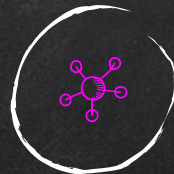




# PROTOCOLE HTTP

## Les réponses





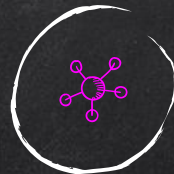
# API REST URI

(Uniform Resource Identifier)

*Pour chaque ressource nous aurons 5 URI (CRUD) uniques*

Exemple pour une ressource de livres :

- **(GET)** `http://site.com/api/books` => Renvoie la liste de tous les livres.
- **(GET)** `http://site.com/api/books/{identifiant-unique}` => Renvoie un livre spécifique.
- **(POST)** `http://site.com/api/books` => Ajouter un livre.
- **(PUT)** `http://site.com/api/books/{identifiant-unique}` => Modifier livre spécifique.
- **(DELETE)** `http://site.com/api/books/{identifiant-unique}` => Supprimer un livre.



# API REST

## LES BON CODES DE RETOUR

### LISTE DES CODES DE SUCCES :

- **200 OK**: Tout s'est bien passé
- **201 Created** : La création de la ressource s'est bien passée (il n'est pas rare que les attributs de la nouvelle ressource soient aussi renvoyés dans la réponse. Dans ce cas, l'URL de cette ressource nouvellement créée est ajouté via un header Location )
- **204 No content**: Même principe que pour la 201, sauf que cette fois-ci, le contenu de la ressource nouvellement créée ou modifiée n'est pas renvoyée en réponse
- **304 Not modified**: Le contenu n'a pas été modifié depuis la dernière fois qu'elle a été mise en cache

### LISTE DES CODES D'ERREURS :

- **400 Bad request**: La demande n'a pas pu être traitée correctement
- **401 Unauthorized**: L'authentification a échoué
- **403 Forbidden**: L'accès à cette ressource n'est pas autorisé
- **404 Not found**: La ressource n'existe pas
- **405 Method not allowed**: La méthode HTTP utilisée n'est pas traitable par l'API
- **406 Not acceptable**: L'API est dans l'incapacité de fournir le format demandé par les en têtes Accept. Par exemple, le client demande un format (XML par exemple) et l'API n'est pas prévue pour générer du XML
- **500 Server error**: Le serveur a rencontré un problème.

Choix du modèle

Configurations

Choix du template

## Créer un projet

## Modèles de projet récents

- Application web ASP.NET Core C#
- Application WPF (.NET Framework) C#
- WPF App (.NET Core) C#
- Windows Forms App (.NET Core) C#
- Bibliothèque de classes (.NET Core) C#
- Bibliothèque de classes (.NET Framework) C#
- Application Windows Forms (.NET Framework) C#
- Bibliothèque de classes (.NET Standard) C#

Tout effacer

C#

Windows

Web



## Application web ASP.NET Core

Modèles de projet permettant de créer des applications web ASP.NET Core et des API web pour Windows, Linux et macOS à l'aide du .NET Core ou du .NET Framework. Créez des applications web avec Razor Pages, MVC ou des SPA (applications monopages) via Angular, React ou React + Redux.

C#

Linux

macOS

Windows

Cloud

Service

Web



## Application Blazor

Modèles de projet pour la création d'applications Blazor qui s'exécutent sur le serveur dans une application ASP.NET Core ou dans le navigateur de WebAssembly. Vous pouvez utiliser ces modèles pour générer applications web ayant des IU (interfaces utilisateur) dynamiques riches.

C#

Linux

macOS

Windows

Cloud

Web



## Service gRPC

Modèle de projet pour la création d'un service gRPC ASP.NET Core à l'aide du .NET Core.

C#

Linux

macOS

Windows

Cloud

Service

Web



## Bibliothèque de classes Razor

Modèle de projet pour créer une bibliothèque de classes Razor.

C#

Linux

macOS

Windows

Bibliothèque

Web



## Projet de test NUnit (.NET Core)

Projet qui contient des tests NUnit qui peuvent s'exécuter sur .NET Core sur Windows, Linux et MacOS.

C#

Linux

macOS

Windows

Bureau

Test

Web



## Application web ASP.NET (.NET Framework)

Modèles de projet permettant de créer des applications ASP.NET. Vous pouvez créer des applications ASP.NET Web Forms, MVC, ou API web, et ajouter de nombreuses autres fonctionnalités ASP.NET.

Retour

Suivant

Modèle API WEB  
.net Core

Choix du modèle

Configurations

Choix du template

## Configurer votre nouveau projet

Application web ASP.NET Core C# Linux macOS Windows Cloud Service Web

Nom du projet

WebApplication1

Emplacement

C:\Users\fb\Source\Repos

Nom de la solution ⓘ

WebApplication1

☐ Placer la solution et le projet dans le même répertoire

**Attention**  
décocher la case

Nom de l'API

Emplacement du  
projet

Nom du projet

Retour

Créer



Choix du modèle

Configurations

Choix du template

Template API

## Créer une application web ASP.NET Core

.NET Core ASP.NET Core 3.1

**Vide**  
Modèle de projet vide pour la création d'une application ASP.NET Core. Ce modèle n'a aucun contenu.

**API**  
Modèle de projet permettant de créer une application ASP.NET Core avec un exemple de contrôleur pour un service HTTP RESTful. Vous pouvez également utiliser ce modèle pour les vues et contrôleurs ASP.NET Core MVC.

**Application web**  
Modèle de projet permettant de créer une application ASP.NET Core avec un exemple de contenu ASP.NET Core Razor Pages.

**Application web (Model-View-Controller)**  
Modèle de projet permettant de créer une application ASP.NET Core avec des exemples de vues et de contrôleurs ASP.NET Core MVC. Vous pouvez également utiliser ce modèle pour les services HTTP RESTful.

**Angular**  
Modèle de projet permettant de créer une application ASP.NET Core avec Angular.

**React.js**  
Modèle de projet permettant de créer une application ASP.NET Core avec React.js.

**React.js et Redux**  
Modèle de projet pour créer une application ASP.NET Core avec React.js et Redux.

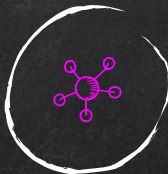
**Authentification**  
Aucune authentification  
[Changer](#)

**Avancé**  
☒ Configurer pour HTTPS  
☐ Activer le support de Docker  
(Nécessite [Docker Desktop](#))  
Linux

Auteur : Microsoft  
Source : .NET Core 3.1.4

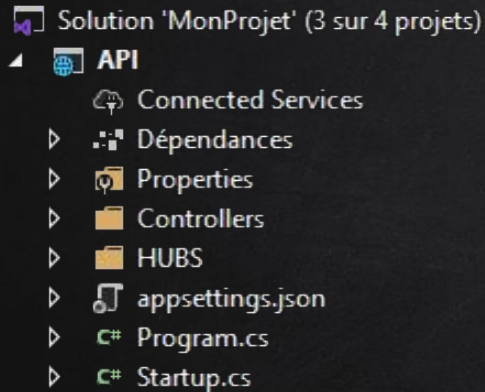
[Obtenir des modèles de projet supplémentaires](#)

Retour Créer

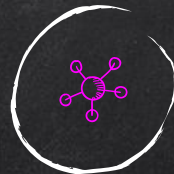


# API REST

## Vue de la structure du projet



| <i>Label</i>     | <i>Description</i>                                       |
|------------------|----------------------------------------------------------|
| Dépendances      | Liste des librairies utilisées par le projet             |
| Properties       | Propriétés de lancement du projet                        |
| Controllers      | Contient tous les contrôleurs de l'API REST              |
| HUBS             | Contient tous les HUB Signal-R                           |
| appsettings.json | Fichier de configuration de l'application en JSON        |
| Program.cs       | Classe main qui encapsule le startup dans un serveur WEB |
| Startup.cs       | Classe de lancement du projet                            |



# API REST

## Les contrôleurs

### Principes

La classe ControllerBase permet de regrouper les méthodes de l'interface WEB

### Pattern des contrôleurs

```
[ApiController] // Contrôleur REST
[Route("/api/[controller]")] // URI de base : "/api/nomdelaressource"
public class NomDeLaRessourceController : ControllerBase {
    // Méthodes du controller
    // ...
}
```

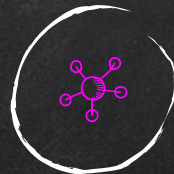
| Décorateurs de controller                                                                                              | Description                                                                                                                                                                                                                              |
|------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p>ApiController</p> <p><u>Exemple:</u><br/>[ApiController]</p>                                                        | <p>Définit que la classe est un contrôleur d'API</p>                                                                                                                                                                                     |
| <p>Route</p> <p><u>Exemple:</u><br/>[Route("/api/[controller]")]</p>                                                   | <p>La route de base associée:<br/> <u>Exemple pour la classe BooksController:</u><br/> <a href="http://site.com/api/books">http://site.com/api/books</a><br/> <u>Remarque:</u><br/> [Controller] =&gt; BooksController =&gt; "books"</p> |
| <p>Produces</p> <p><u>Exemple:</u><br/>[Produces(MediaTypeNames.Application.Json)]</p>                                 | <p>Définit le format que l'api doit produire (Json, Xml, ...)</p>                                                                                                                                                                        |
| <p>Consumes</p> <p><u>Exemple:</u><br/>[Consumes(MediaTypeNames.Application.Json, MediaTypeNames.Application.Xml)]</p> | <p>Définit le format que l'api doit consommer (Json, Xml,...)</p>                                                                                                                                                                        |

| Décorateurs d'arguments                                                 | Description                                                                                                              |
|-------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| FromHeader                                                              | Argument qui vient du header de la requête HTTP                                                                          |
| FromRoute<br><u>Exemple:</u><br><code>[FromRoute] int? id</code>        | Argument qui vient de l'url de la requête<br><u>Exemple:</u><br><code>http://site.com/api/books/{id}</code>              |
| FromQuery<br><u>Exemple:</u><br><code>[FromQuery] string? search</code> | Argument qui vient des paramètres de la requête<br><u>Exemple:</u><br><code>http://site.com/api/books?search=test</code> |
| FromBody                                                                | Désérialisation du body de la requête dans l'argument                                                                    |
| FromForm                                                                | Désérialisation du form data de la requête dans l'argument                                                               |
| FromServices                                                            | Injecter un service particulier avec l'IOC                                                                               |





| Décorateurs de méthodes                                                                                                                                                                     | Description                                                  |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------|
| <b>HttpGet</b><br><u>Exemples:</u><br><code>[HttpGet()]</code><br><code>[HttpGet("{id}")]</code>                                                                                            | Définit une méthode GET et prend en argument la route        |
| <b>HttpPost</b><br><u>Exemple:</u><br><code>[HttpPost("{id}")]</code>                                                                                                                       | Définit une méthode POST et prend en argument la route       |
| <b>HttpPut</b><br><u>Exemple:</u><br><code>[HttpPut("{id}")]</code>                                                                                                                         | Définit une méthode PUT et prend en argument la route        |
| <b>HttpDelete</b><br><u>Exemple:</u><br><code>[HttpDelete("{id}")]</code>                                                                                                                   | Définit une méthode DELETE et prend en argument la route     |
| <b>ProducesResponseType</b><br><u>Exemple:</u><br><code>[ProducesResponseType(StatusCodes.Status201Created)]</code><br><code>[ProducesResponseType(StatusCodes.Status400BadRequest)]</code> | Définit les codes qui peuvent être renvoyés par une méthode. |



# API REST

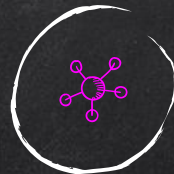
## GET

### Quand l'utiliser la méthode GET

La méthode **GET** permet de récupérer une ou plusieurs ressources.

### Pattern de la méthode

```
[HttpGet()]  
public async Task<ActionResult<List<TypeDeRessource>>> GETNomDeLaRessource() {  
  
}  
  
[HttpGet("{id}")]  
[ProducesResponseType(StatusCodes.Status200OK)]  
[ProducesResponseType(StatusCodes.Status404NotFound)]  
public async Task<IActionResult> GETNomDeLaRessourceById([FromRoute] int id) {  
  
}
```

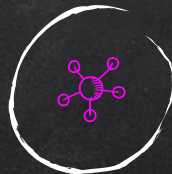


# API REST

## EXEMPLES DE MÉTHODES GET

### Récupération de la liste des livres

```
// Controller = BooksController
// URL associées :
// /api/books?page=1&count=20
// /api/books?page=1
// /api/books?count=20
// /api/books
[HttpGet()]
public async Task<ActionResult<List<Book>>> GetBooks([FromQuery]int? page, [FromQuery]int? count) {
    return await _libraryService.GetAllBooks(page, count);
}
```

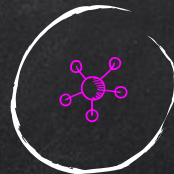


# API REST

## EXEMPLES DE MÉTHODES GET

### Récupération d'un livre par son identifiant unique

```
// URL associées :  
// /api/books/8  
[HttpGet("{id}")]  
[ProducesResponseType(StatusCodes.Status200OK)]  
[ProducesResponseType(StatusCodes.Status404NotFound)]  
public async Task<ActionResult> GetBookById([FromRoute] int id)  
{  
    Book book = await _libraryService.GetBookById(id);  
    if(book == null)  
    {  
        return NotFound(); // StatusCode = 404  
    }  
    else  
    {  
        return Ok(book); // StatusCode = 200  
    }  
}
```



# API REST

## POST

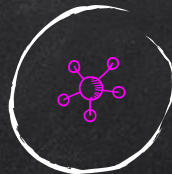
### Quand l'utiliser la méthode POST

La méthode **POST** permet d'ajouter de nouvelles ressources.

### Pattern de la méthode

```
[HttpPost()]  
[ProducesResponseType(StatusCodes.Status201Created)]  
[ProducesResponseType(StatusCodes.Status400BadRequest)]  
public async Task<IActionResult> CreateNomDeLaRessource([FromBody] TypeRessource Ressource)  
{  
  
}
```



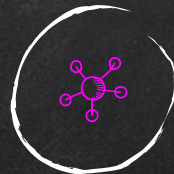


# API REST

## EXEMPLE DE MÉTHODE POST

### Création d'un livre

```
// URL associées :  
// /api/books  
[HttpPost()]  
[ProducesResponseType(StatusCodes.Status201Created)]  
[ProducesResponseType(StatusCodes.Status400BadRequest)]  
public async Task<IActionResult> CreateBook([FromBody] Book book)  
{  
    // Ajout du livre avec la bll server  
    Book newBook = await _libraryService.CreateBook(book);  
    if(newBook != null)  
    {  
        // Créer une redirection vers GetBookById(newBook.BookId);  
        return CreatedAtAction(nameof(GetBookById), new { id = newBook.BookId }, newBook);  
    }  
    else  
    {  
        // Retourne un code 400 Bad Request  
        return BadRequest();  
    }  
}
```



# API REST

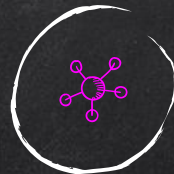
## PUT

### Quand l'utiliser la méthode PUT

La méthode **PUT** permet de modifier une ressource.

### Pattern de la méthode

```
[HttpPut("{id}")]  
[ProducesResponseType(StatusCodes.Status200OK)]  
[ProducesResponseType(StatusCodes.Status400BadRequest)]  
[ProducesResponseType(StatusCodes.Status404NotFound)]  
public async Task<IActionResult> ModifyNomDeLaRessource([FromRoute] int id, [FromBody] TypeRessource Ressource)  
{  
}
```

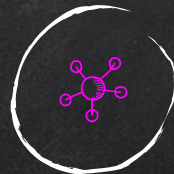


# API REST

## EXEMPLE DE MÉTHODE PUT

### Modification d'un livre

```
// /api/books/8
[HttpPut("{id}")]
[ProducesResponseType(StatusCodes.Status200OK)]
[ProducesResponseType(StatusCodes.Status400BadRequest)]
[ProducesResponseType(StatusCodes.Status404NotFound)]
public async Task<IActionResult> ModifyBook([FromRoute] int id, [FromBody] Book book)
{
    if(id != book.BookId)
    {
        // Retourne un code 400 Bad Request
        return BadRequest();
    }
    else
    {
        Book bookModified = await _controllerService.ModifyBook(book);
        if(bookModified != null)
        {
            // Renvoie la ressource modifiée
            return Ok(bookModified);
        }
        else
        {
            // Renvoie un code 404: la ressource est introuvable
            return NotFound();
        }
    }
}
```



# API REST

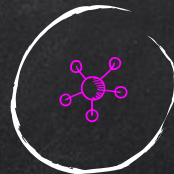
## DELETE

### Quand l'utiliser la méthode DELETE

La méthode **DELETE** permet de supprimer une ressource.

### Pattern de la méthode

```
[HttpDelete("{id}")]  
[ProducesResponseType(StatusCodes.Status204NoContent)]  
[ProducesResponseType(StatusCodes.Status404NotFound)]  
public async Task<IActionResult> DeleteNomDeLaRessource([FromRoute] int id)  
{  
}
```



# API REST

Suppression d'un livre

## EXEMPLE DE MÉTHODE DELETE

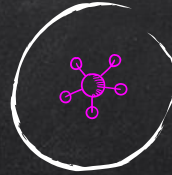
```
[HttpDelete("{id}")]
[ProducesResponseType(StatusCodes.Status204NoContent)]
[ProducesResponseType(StatusCodes.Status404NotFound)]
public async Task<IActionResult> DeleteBook([FromRoute] long id)
{
    if (await _libraryService.DeleteBook(id))
    {
        // Renvoie un code 204 aucun contenu
        return NoContent();
    }
    else
    {
        // Renvoie un code 404: la ressource est introuvable
        return NotFound();
    }
}
```



## Book

Business Object

- ☐ Identifiant unique
- ☐ ISBN
- ☐ Titre
- ☐ Description



API REST  
TP1

*Créer une API de gestion de livre avec l'interface suivante :*

- **(GET)** `http://site.com/api/books` => Renvoie la liste de tous les livres.
- **(GET)** `http://site.com/api/books/{identifiant-unique}` => Renvoie un livre spécifique.
- **(POST)** `http://site.com/api/books` => Ajouter un livre.
- **(PUT)** `http://site.com/api/books/{identifiant-unique}` => Modifier livre spécifique.
- **(DELETE)** `http://site.com/api/books/{identifiant-unique}` => Supprimer un livre.